

University of Ruhuna
Faculty of Engineering
Department of Electrical and Information Engineering

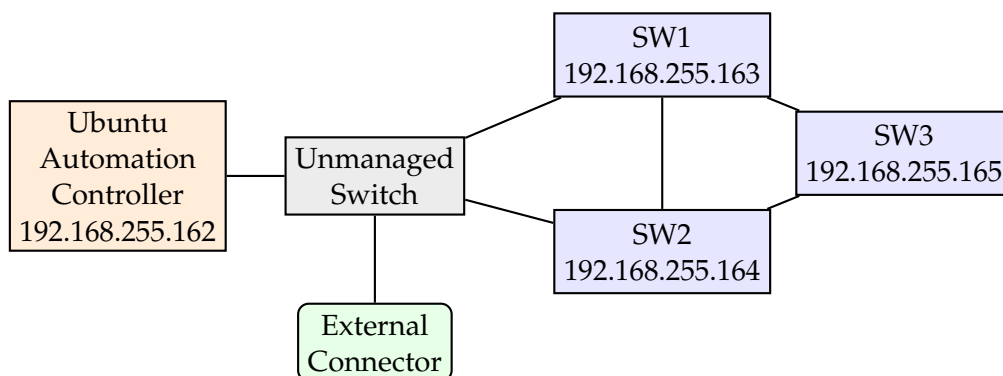
Network Automation with Ansible

A Hands-on Workshop Guide

Module: EE8203/EC8205 - Design and Management of Data Networks

Simulation Platform: Cisco Modeling Labs (CML)

Automation Tool/ Configuration Management Tool: Ansible



GitHub Repository (code & configs):
<https://github.com/saahith-networking/CML-Test.git>

Eng. R. Saahith Ahamed
Lecturer
Department of Electrical and Information Engineering
(Former Data Communication Engineer at Huawei)

Contents

1	Introduction	2
1.1	Learning Objectives	2
2	Lab Topology	3
2.1	Addressing Table	3
3	Environment Setup on the Ubuntu Controller	4
3.1	Create and Activate a Python Virtual Environment	4
3.2	Install Required Python Packages	4
3.3	Install the Cisco IOS Collection	4
3.4	Enable Colored Output (Optional)	4
4	Enabling SSH on the Cisco IOS Switches	5
5	Building the Ansible Inventory	7
6	The <code>ansible.cfg</code> Configuration File	8
6.1	Why Three Separate Files?	8
7	Running Ad-hoc Ansible Commands	8
7.1	What is an Ad-hoc Command?	8
7.2	Connectivity Test	9
7.3	Running <code>show</code> Commands	9
7.4	More Ad-hoc Examples	9
8	Writing and Running a Playbook	10
8.1	Example 1: Configuring a Login Banner	10
8.2	Example 2: Gathering Device Information	11
9	Version Control with Git and GitHub	12
9.1	Cloning the Repository	12
9.2	Basic Git Workflow	12
10	Appendix: Command Quick Reference	13

1 Introduction

This guide accompanies the hands-on Ansible workshop conducted for the EE8203/EC8205 - Design and Management of Data Networks module. It documents, in a repeatable step-by-step format, the environment setup and command sequences used during the session, so that you can reproduce the lab independently or revisit any step you may have missed.

The workshop covered:

- Setting up an isolated Python environment for network automation
- Installing and configuring Ansible together with the `cisco.ios` collection and Netmiko
- Enabling SSH access on Cisco IOS switches simulated in Cisco Modeling Labs (CML)
- Running ad-hoc Ansible commands against network devices
- Writing and executing an Ansible playbook
- Managing lab code and configuration files using Git and GitHub

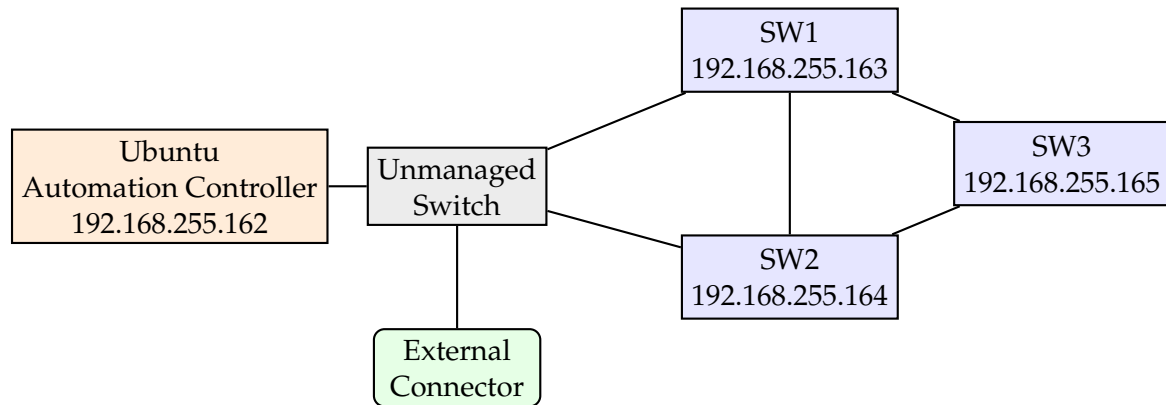
1.1 Learning Objectives

By the end of this workshop, students should be able to:

1. Explain the role of Ansible in Network Automation and how it differs from traditional CLI-based configuration.
2. Set up a controller machine (Ubuntu) with a Python virtual environment for Ansible.
3. Build an Ansible inventory file describing a group of network devices.
4. Execute ad-hoc Ansible modules against Cisco IOS switches over SSH.
5. Write a simple Ansible playbook to automate repetitive configuration tasks.
6. Track lab work using version control (Git/GitHub).

2 Lab Topology

The lab was built in **Cisco Modeling Labs (CML)** using three Cisco IOS switches connected to an unmanaged switch, which in turn connects to the Ubuntu automation controller. An external connector was attached to the unmanaged switch to bridge the CML lab network to the host network, allowing the Ubuntu controller to reach the switch management interfaces over SSH.



2.1 Addressing Table

Device	Management IP	Role
Ubuntu VM	192.168.255.162	Ansible control node (automation controller)
SW1	192.168.255.163	Cisco IOS switch, SSH enabled
SW2	192.168.255.164	Cisco IOS switch, SSH enabled
SW3	192.168.255.165	Cisco IOS switch, SSH enabled
Unmanaged Switch	–	Layer-1 connectivity between controller and CML nodes

3 Environment Setup on the Ubuntu Controller

All automation commands are run from the Ubuntu VM (192.168.255.162), which acts as the Ansible control node.

3.1 Create and Activate a Python Virtual Environment

Using a virtual environment keeps the Ansible installation isolated from the system Python packages.

```
python3 -m venv venv
source venv/bin/activate
```

Once activated, the shell prompt will be prefixed with (venv), confirming that the virtual environment is active.

3.2 Install Required Python Packages

With the virtual environment active, upgrade pip and install Ansible and Netmiko:

```
pip install --upgrade pip
pip install ansible
pip install netmiko
```

3.3 Install the Cisco IOS Collection

Ansible modules for Cisco IOS devices (e.g. `ios_command`) are shipped as a separate collection and must be installed before use:

```
ansible-galaxy collection install cisco.ios
```

To confirm the collection installed correctly and to list the available Cisco IOS modules:

```
ansible-doc -l | grep -i cisco.ios
```

3.4 Enable Colored Output (Optional)

By default, Ansible output in some terminals/CI environments is not colorized. Force colored output with:

```
export ANSIBLE_FORCE_COLOR=true
```

4 Enabling SSH on the Cisco IOS Switches

Before Ansible can manage the switches, SSH must be enabled on each device in CML. The following configuration was applied on **SW1**, **SW2**, and **SW3** (repeat with the appropriate hostname and IP for each switch).

The commands must be applied in the order shown below – the hostname and domain name have to be set *before* the RSA key is generated, and the VTY lines must reference login local so that SSH accepts the local username/password.

1. Enter global configuration mode

```
enable
configure terminal
```

2. Set the hostname (required before key generation)

```
hostname SW1
```

3. Set the IP domain name (required before key generation)

```
ip domain-name saahith-netlab.com
```

4. Create a local user account with privilege level 15 (full access)

```
username {username} privilege 15 secret {secret}
```

Replace {username} and {secret} with the actual credentials used for the workshop (e.g. username admin privilege 15 secret Cisco@123).

5. Generate the RSA key pair used by SSH (this step is what actually turns SSH on)

```
crypto key generate rsa modulus 2048
```

Use a modulus of **2048** bits (the minimum recommended for SSH version 2). If a key already exists, IOS will ask whether to replace it – type yes.

6. Force SSH version 2 (optional but recommended)

```
ip ssh version 2
```

7. Configure the VTY (remote-access) lines to accept SSH using the local username database

```
line vty 0 4
login local
transport input ssh
exit
```

8. Configure the management interface facing the unmanaged switch / external connector

```
interface <management-interface>
ip address 192.168.255.163 255.255.255.0
no shutdown
exit
```

9. Save the configuration

```
end
write memory
```

Notes:

- Steps 2 and 3 (hostname, domain name) must be completed *before* Step 5 (crypto key generate rsa) – IOS uses the hostname and domain name to build the key's identity, and will refuse to generate a usable key without them.
- login local in Step 7 tells the VTY lines to authenticate against the local username database created in Step 4, instead of expecting a plain-text line password.
- Replace <management-interface> with the actual interface connected toward the unmanaged switch/external connector in your CML topology.
- Repeat this entire sequence on SW2 (192.168.255.164) and SW3 (192.168.255.165), changing the hostname and IP address accordingly.
- Verify connectivity from the Ubuntu controller before proceeding:
ssh {username}@192.168.255.163

5 Building the Ansible Inventory

Ansible needs an inventory file describing the devices to manage, grouped logically. Create a file named `hosts.ini`:

```
[switches]
192.168.255.163
192.168.255.164
192.168.255.165

[switches:vars]
ansible_user=admin
ansible_password=<your-password>
ansible_connection=network_cli
ansible_network_os=ios
ansible_port=22
```

Key points:

- The group name `[switches]` is referenced directly in ad-hoc commands and playbooks.
- `ansible_connection=network_cli` tells Ansible to use the network CLI connection plugin (SSH-based) rather than the default `ssh` connection plugin used for Linux hosts.
- `ansible_network_os=ios` selects the correct platform plugin from the `cisco.ios` collection installed earlier.

6 The `ansible.cfg` Configuration File

Alongside the inventory, an `ansible.cfg` file was created in the same project folder to control Ansible's own behaviour. Ansible automatically looks for this file in the current working directory before falling back to system-wide or user-wide defaults.

```
[defaults]
host_key_checking = False
deprecation_warnings = False
```

What each setting does:

- `host_key_checking = False` – disables SSH host-key verification against `known_hosts`. In a CML lab the switches' management IPs are frequently reused/rebuilt as topologies are reset, which would otherwise cause SSH to refuse the connection with a *"REMOTE HOST IDENTIFICATION HAS CHANGED"* warning. This setting removes that friction for lab use; it should **not** be disabled in a production network.
- `deprecation_warnings = False` – suppresses Ansible's deprecation notices in the console output, keeping the terminal output focused on actual task results during the workshop rather than library warnings.

6.1 Why Three Separate Files?

By the end of the setup, the project folder contains three files that work together, each with a distinct responsibility:

File	Necessity
<code>ansible.cfg</code>	Controls how Ansible itself behaves (host-key checking, warning verbosity, and other engine-level settings). Read automatically once it exists in the project directory – no <code>-i</code> or command-line flag needed.
<code>hosts.ini</code>	The inventory: tells Ansible <i>which</i> devices exist, how to reach them (IP, connection type, platform), and how to group them (e.g. <code>[switches]</code>). Without it, Ansible has no devices to target.
<code>cisco.yml</code>	The playbook: describes <i>what</i> Ansible should actually do once connected to the devices in the inventory – the ordered list of tasks/modules to run.

In short: `ansible.cfg` configures the tool, `hosts.ini` tells it *where* to connect, and `cisco.yml` tells it *what* to do there.

7 Running Ad-hoc Ansible Commands

7.1 What is an Ad-hoc Command?

An **ad-hoc command** is a single Ansible command typed directly at the terminal using the `ansible` CLI tool (as opposed to `ansible-playbook`, which runs a saved YAML file). It runs exactly *one* module, exactly *once*, against a target group from the inventory, and then exits – nothing is saved for later reuse.

Ad-hoc commands are best suited to quick, throwaway tasks: checking that a device is reachable, pulling a single piece of show output, or testing a module's syntax before committing it to a playbook. Anything that needs to be repeated, ordered, or shared with others belongs in a playbook instead (Section 8).

The general form used throughout this workshop is:

```
ansible <group> -m <module> -a "<module arguments>" -i hosts.ini
```

where <group> is a group from the inventory (e.g. switches), -m selects the module to run, and -a passes arguments to that module.

7.2 Connectivity Test

```
ansible switches -m ping -i hosts.ini
```

A successful response returns "ping": "pong" for each switch in the [switches] group, confirming that Ansible can reach and authenticate to all three devices.

7.3 Running show Commands

The `cisco.ios.ios_command` module executes one or more exec-mode commands and returns their output.

```
ansible switches -m cisco.ios.ios_command -a "commands='show version'" -i hosts.ini
```

```
ansible switches -m cisco.ios.ios_command -a "commands='show vlan brief'" -i hosts.ini
```

Tip: Quoting matters. Keep the command string inside single quotes nested within the double-quoted -a argument, exactly as shown above, to avoid shell parsing errors.

7.4 More Ad-hoc Examples

1. Checking IP interface status

```
ansible switches -m cisco.ios.ios_command -a "commands='show ip interface brief'" -i hosts.ini
```

2. Targeting a single device instead of the whole group

```
ansible 192.168.255.163 -m cisco.ios.ios_command -a "commands='show running-config'" -i hosts.ini
```

Ansible accepts an individual host name/ip address from the inventory (here, 192.168.255.163) in place of a group name, which is handy when only one device needs checking.

3. Gathering structured device facts

```
ansible switches -m cisco.ios.ios_facts -i hosts.ini
```

Unlike `ios_command`, which returns raw CLI text, `cisco.ios.ios_facts` returns structured data (hostname, model, serial number, IOS version, interface list, and more) that downstream playbooks or scripts can consume directly.

4. Running more than one command at once

```
ansible switches -m cisco.ios.ios_command -a "commands='show clock','show inventory'" -i hosts.ini
```

Multiple commands can be comma-separated inside the same `commands=` argument, each device's output is returned for both commands in a single run.

8 Writing and Running a Playbook

While ad-hoc commands are handy for quick checks, playbooks let you describe a repeatable, version-controlled set of tasks. Both examples below were saved to the same file, `cisco.yml`, during the workshop.

8.1 Example 1: Configuring a Login Banner

This was the first playbook used in the session – a simple, single-task example that pushes a login banner to every switch in the `[switches]` group using the `cisco.ios.ios_banner` module.

```
---
- name: Our first playbook
  hosts: switches
  become: yes

  tasks:
    - name: Configure the login banner
      cisco.ios.ios_banner:
        banner: login
        text: |
          Welcome to Network Automation with Ansible!!!
          This is EE8203 DMDN
        state: present
```

Key points:

- `become: yes` tells Ansible to enter privileged (enable) mode on the switch before running the task - equivalent to typing `enable` manually.
- `banner: login` configures the banner shown *before* the login prompt (as opposed to `motd` or `exec` banners).
- `state: present` ensures the banner is added/updated, setting it to `absent` would remove an existing banner instead.

8.2 Example 2: Gathering Device Information

The second example queries each switch for its version and VLAN information and displays the results in the terminal.

```
---
- name: Basic Cisco IOS information gathering
  hosts: switches
  gather_facts: no

  tasks:
    - name: Show version
      cisco.ios.ios_command:
        commands:
          - show version
      register: version_output

    - name: Show VLAN brief
      cisco.ios.ios_command:
        commands:
          - show vlan brief
      register: vlan_output

    - name: Display version output
      debug:
        var: version_output.stdout_lines

    - name: Display VLAN output
      debug:
        var: vlan_output.stdout_lines
```

Run either playbook with:

```
ansible-playbook cisco.yml -i hosts.ini
```

Extending the playbook: Once comfortable with these two examples, the same pattern (using `cisco.ios.ios_config` for arbitrary configuration lines) can push further changes, such as creating VLANs or adjusting interface settings across all three switches in a single run.

9 Version Control with Git and GitHub

All configuration files, inventories, and playbooks used in this workshop are tracked in the following GitHub repository:

<https://github.com/saahith-networking/CML-Test.git>

9.1 Cloning the Repository

```
git clone https://github.com/saahith-networking/CML-Test.git
cd CML-Test
```

9.2 Basic Git Workflow

```
git status
git add hosts.ini cisco.yml
git commit -m "Add inventory and playbook for switch automation lab"
git push origin main
```

Why version control matters here: As you iterate on inventories and playbooks across labs, Git preserves a history of every change, makes it easy to roll back a broken playbook, and lets you compare your work against the reference copy in the repository above.

10 Appendix: Command Quick Reference

```
# Environment setup
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install ansible
pip install netmiko
ansible-galaxy collection install cisco.ios

# Enable colored output
export ANSIBLE_FORCE_COLOR=true

# List available Cisco IOS modules
ansible-doc -l | grep -i cisco.ios

# ansible.cfg (place in the project folder)
# [defaults]
# host_key_checking = False
# deprecation_warnings = False

# Connectivity test
ansible switches -m ping -i hosts.ini

# Ad-hoc show commands
ansible switches -m cisco.ios.ios_command -a "commands='show version'" -i
    hosts.ini
ansible switches -m cisco.ios.ios_command -a "commands='show vlan brief'" -i
    hosts.ini

# Run a playbook
ansible-playbook cisco.yml -i hosts.ini

# Git workflow
git clone https://github.com/saahith-networking/CML-Test.git
git add .
git commit -m "message"
git push origin main
```

End of Workshop Guide - EE8203/EC8205: Design and Management of Data Networks